THE UNIVERSITY OF SYDNEY

MTRX3760 - Lab 2
Robot Simulator

This assignment contributes 10% towards your final mark. It must be completed in groups of two. Use the Canvas groups functionality to build your team and complete a single submission. Groups of one or three are not permitted except with prior approval from the unit coordinator. All group members must be enrolled in the same practical section.

This assignment is due before the start of your allocated lab session in two weeks, i.e. before the start of the Week 6 lab session for Friday labs, and before the start of the Week 7 lab session for Monday labs. Late assignments will be subject to the University's late submission policy. Plagiarism will be dealt with in accordance with the University of Sydney plagiarism policy. Submissions will not be accepted more than a week after the due date.

Assignments will be assessed based on the following components. Incomplete submissions will not be graded. A1 and A2 also carry a functionality requirement: to be graded, your code must build, run, and complete both tasks to the standard set by the Minimum Functionality Hurdle (see A2). A submission that does not clear the hurdle earns no marks for A1 and A2.

- **Report**: Submit a .pdf report using the class Canvas site. The report can be very simple, and needs only directly address the questions laid out in the assignment. The cover page for your report should include your group members' SIDs and practical section, do not include names.
- **Code appendix**: The appendix of your report must contain a printout in text format of all source code written for the assignment. File header comments and proper formatting will be critical to making this section readable. Inclusion as images is unacceptable.
- **Code**: Submit your code (code only, no binaries) in a .zip file via the canvas site.

You may find libreoffice writer or google or microsoft's online tools useful for preparing your report. For printing out and appending code to your report, you can use:

```
find . -name "*.cpp" -o -name "*.h" | sort -V | xargs enscript -r --color=1 -C
-Ecpp -fCourier8 -o - | ps2pdf - code.pdf

pdfunite <report name>.pdf code.pdf <concatenated report name>.pdf
```

The first command recursively finds `.cpp` and `.h` files and formats them as a pdf called `code.pdf`. The second command concatenates the code to an existing pdf file.

This lab should be completed with concepts and language features taught in **Weeks 1 through 4 only**.

This lab has the following goals, your work will be evaluated with these in mind:

1. To build your understanding of how a system of moderate complexity can be designed and implemented in an object-oriented manner,
2. To give you an opportunity to demonstrate the object-oriented system design and C++ syntax, structure, and style practices you have learned so far,
3. To exercise design thinking and decision-making when faced with the many open design choices typical of a practical engineering problem, and
4. To practice working well in a team.

Begin by downloading the example code provided on Canvas. The readme file describes the purpose of each file, and includes instructions for installing the required raylib library. You may modify and adapt the provided code however you wish, but you must use raylib, and all raylib access must go through a CRender class that wraps it behind a C++ interface.

A0. Team Setup [10 marks]

Read through the entire lab to get a feel for its requirements. Before getting started, establish your team's structure. In your report write down each of your team member's responses to the following questions:

- What is each team member's main strength relevant to this lab?
- What is each team member's main area for improvement relevant to this lab?
- How will you arrive at decisions as a team, including how you will resolve disagreements?
- How will you keep on schedule?
- Will you use revision control and code review? Why / why not?

THE UNIVERSITY OF SYDNEY

A1: Wall Follower

Build a simulation of a robot that drives around a room, following the wall on its right hand side, until it has driven all the way around. A test room is supplied in `SimpleWalls.map`. All walls are straight, there are no curves, and the room is not a maze. The map file also specifies the robot's starting position.

Your robot must meet the following specification. These constraints keep the task tractable, and a submission that departs from them is not meeting the requirements:

- The robot is a disc of radius 15 units. It is drawn as a circle of that radius with a line indicating the direction it is facing.
- The robot carries exactly two range sensors, both aimed to the right hand side. One faces directly to the robot's right, at 90 degrees; the other faces forward and to the right, at 45 degrees. Each reports the distance from the robot to the first wall its ray meets.
- The robot is driven by two independently controlled wheels, one either side.
- Each update advances the simulation by a fixed amount of simulated time. Do not drive the simulation from real elapsed time.
- The robot leaves a trail behind it showing its trajectory, and the trail remains visible for the whole run.
- The simulation runs long enough for the robot to make it all the way around the room.
- Each time the robot collides with a wall, the collision is reported to the console.
- At the end of the run, print a short summary to the console, including the number of updates completed and the total number of collisions.
- Your program must clear the Minimum Functionality Hurdle, below.

Constraints

- You must follow the "encapsulate everything" design approach described in-lecture. Recall this is loose terminology describing the concept that locality should be maximised. So all consts, enums, and certainly all data and functions should be embedded inside classes, but not necessarily private.
- You should use multiple files, generally one header file for every major class.

In your report include:

- A UML class diagram for your design, following the unit's simplified standard. Do not show member variables or member functions.
- A screenshot taken at the end of a run completing the robot's trail all the way around the room in `SimpleWalls.map`.
- Your console output copy/pasted into your report, showing any collisions.

THE UNIVERSITY OF SYDNEY

A2: Line Follower

Complete A1 first, and keep a copy of it. You must submit full code for both A1 and A2.

Add a second robot to the simulation. This one follows a line marked on the floor. A test line is supplied in `SimpleLine.map`. It is a set of straight segments forming a closed loop, along with a robot starting position and heading. Treat the line as 5 units wide for sensing.

Both robots run in the same room at the same time, in the same program. They do not interact, and neither needs to avoid the other or the other's line.

The line following robot meets the same specification as the wall follower, with one difference:

- In place of the two range sensors, it carries exactly two line sensors, one over the line and one to the side of the line. Each reports whether the floor directly beneath it is line or not.
- Your program must clear the Minimum Functionality Hurdle, below.

Everything else stands: two wheels, fixed timestep, drawn as a circle with a heading indicator, leaving a trail that remains visible, and run long enough for both robots to make it around their respective maps.

In your report include:

- A UML class diagram for your design, following the unit's simplified standard. Do not show member variables or member functions.
- A screenshot taken at the end of a run showing both robots' trails as they complete `SimpleWalls.map` and `SimpleLine.map`.

Minimum Functionality Hurdle

Your code must build with the standard compile command and run, driving each robot in response to its sensors, steering according to what the sensors report. Both robots need to make it around the circuit, keeping a trail. The wall follower must also mostly avoid the walls: its end-of-run collision count must be 10 or fewer, though zero is the target you're designing toward. A robot that senses and reacts but wanders off and never comes back does not meet the bar. Nor does one that ignores its sensors, for example by driving a fixed path. A submission that does not clear this bar earns no marks for A1 and A2. Once cleared, A1 and A2 are graded on the quality of your design, code, and process, not on how well the robots follow a path.

Design Quality [30 marks]

A1 and A2 will be assessed on design principles and best practices.

Code Quality [30 marks]

A1 and A2 will be assessed on style and coding best practices. Your tutor will annotate the code in the appendix, be sure it's included and legible.

A3: Post-Mortem [10 marks]

Once your programs are designed and implemented, address the following questions as a team:

Reflection on A0, for each team member to answer individually:

- What did each team member contribute to the submission?
- What's one thing you would improve about the Team Setup in future?

Reflection on A1, provide a single response agreed on by both members:

- What's an important aspect of the design of A1 that you had to change in order to make A2 work?
- What's an important aspect of the design of A1 that turned out well when building A2?

Reflection on A2, provide a single response agreed on by both members:

- What's one aspect of the submitted programs' structure that you're especially happy with?
- What's one aspect of the submitted programs' structure you'd most want to improve?
- What's one aspect of the submitted programs' style that you're especially happy with?
- What's one aspect of the submitted programs' style you'd most want to improve?

A4. ROS Tutorials [20 marks]

Complete the ROS 2 tutorials here: https://docs.ros.org/en/jazzy/Tutorials.html . Complete up to and including the Beginner: Client Libraries sections. Be sure to complete the C++ versions of the tutorials.

Augment the Pluginlib example to include a House shape that has a square and has a triangle. The square and triangle should initialise to the same side_length. Test the case in which both have side length 10.0. Even though a house is not strictly speaking a regular polygon, it's okay to have your house inherit from polygon_base::RegularPolygon to complete this task.

Note that as simple as this task is, it does entail some design decisions on your part. Exercise your design decision muscles, and feel free to explain in your report the rationale for any decisions you think might be controversial or unclear but worth consideration as good design.

For the code submission include the completed content of `polygon_plugins.cpp`.

THE UNIVERSITY OF SYDNEY

A5. Noise Bonus [+10 marks]

Make a copy of your A2 program and augment it for this bonus. You will use a random number generator to make your robot simulator more realistic and informative:

- Add a small random offset to the robot's starting position and orientation,
- Add a small random offset to how far each wheel turns in each simulation step, and
- Run 20 of each type of robot simultaneously.

In your report include a screenshot showing most of each type of robot making it around the course. Each robot should follow a visibly different path.

Your submission will be evaluated on its design and code style, and on the quality of the result: a believable spread of trajectories, with the noise visibly perturbing each robot's path while most of each type still make it around. Collisions under noise are not counted, and this bonus does not need to pass the Minimum Functionality Hurdle to attract full marks.

Self-Assessment Bonus

Self-assessment: in your report, estimate your mark for each component of the lab, and your total out of 100. Ignore late penalties, and do not include the self-assessment bonus in your estimate. If your estimated total is within 5 marks of your actual total, you earn the 5 self-assessment marks. The bonus can raise your total above 100, to a maximum of 115.

For example:

```
Functionality   Pass    [Pass/Fail]
A0              7       / 10
Design          23      / 30
Code            22      / 30
A3              8       / 10
A4              20      / 20
A5              4       / +10
Total:          84      / 100
```

THE UNIVERSITY OF SYDNEY

Coding Structure and Style Summary

This summary touches on most of the style and structure concepts covered in the unit up to Week 4. However, when designing and coding you are making many more decisions than are represented here. Practice making considered decisions. See also the Design Checklist from Lab 1.

Lec 1A

- Rather than “using namespace” use the “std::” syntax

Lec 1B

- Informative naming
- Consistent, defensive bracing; one statement per line
- Consistent indentation, spacing, bracing
- Readable variable declarations, one variable per line
- Avoid #defines, they hide bugs well; use consts instead
- Comments are mandatory for good code, see the lecture notes for 6 important spots to check
- Avoid comments that are obvious
- Shape comments & whitespace to highlight structure
- Only use the “this” pointer when necessary

Lec 2A, 2B

- Emphasise parallel structures to help spot bugs
- Avoid repeating code: fewer bugs, less code to maintain
- Avoid multiple exits, these make it harder to debug with breakpoints, couts

Lec 3A, 3B

- Make all data private
- Objects should take care of themselves, present clean interfaces
- Use private functions to break up large functions inside a class
- Use inheritance when “is-a” make sense
- Use composition when “has-a” make sense
- Initialise all variables

Lec 4A, 4B

- Check function arguments are valid
- Avoid magic numbers
- Use correct types for constants
- Eliminate unnecessary #includes
- Pass by (const) reference to stop unnecessary copies
- Const-correct code
- “Encapsulate everything”, a term invented in this unit, see lecture for details